KemdiCode MCP: A Multi-Layer Protocol Server for AI-Assisted Software Engineering

Dawid Irzyk, M.Sc. Eng.

***Abstract*—This paper presents KemdiCode MCP Server, an open-source implementation of the Model Context Protocol (MCP) that extends the specification with 142 specialized tools, a three-layer distributed bus architecture, a persistent cognition subsystem, and multi-provider LLM orchestration across eight providers. The server communicates with IDE clients via JSON-RPC 2.0 over HTTP with Server-Sent Events. We describe in detail the protocol stack comprising three communication layers — L_3 ClusterBus for inter-cluster Redis Pub/Sub signaling with HMAC authentication and bloom-filter deduplication, L_2 DataFlowBus for typed inter-module messaging with Zod-validated envelopes, and L_1 GlobalEventBus for in-process event propagation with chain-depth limiting. We formalize the anti-amplification bridge mechanism, the LLM Magistrale's multi-strategy aggregation with a composite scoring function, and the self-regulating PassController. The cognition subsystem provides cross-session AI self-awareness through eight interconnected stores with reactive event handlers. Experimental validation confirms 559 passing tests and compatibility with four major MCP clients. Version 1.25.0 introduces cluster-bus hardening with circuit breakers, bloom-filter deduplication, and signal-flow backpressure control.**

***Index Terms*—Model Context Protocol, large language models, multi-agent systems, distributed systems, software engineering tools, AI-assisted development**

I. Introduction

MODERN AI-assisted development tools require deep integration between Large Language Models (LLMs) and the developer's working environment. The Model Context Protocol (MCP) [1], introduced by Anthropic in November 2024 and formalized in specification version 2025-11-25 [2], standardizes the interface between AI clients and tool servers using JSON-RPC 2.0 [3] as the transport protocol. However, reference implementations typically provide a narrow set of tools with limited state management and no inter-session persistence.

KemdiCode MCP Server addresses these limitations by extending the MCP paradigm along five dimensions:

(i) *Comprehensive tooling*: 142 tools across 23 categories covering the full software engineering lifecycle, from code analysis and AST-based editing to distributed task management.

(ii) *Persistent cognition*: AI agents retain decision history, confidence calibration, error patterns, and mental models across sessions through Redis-backed stores with configurable TTLs.

(iii) *Distributed execution*: A three-layer bus architecture enables multi-cluster LLM orchestration with quality-driven aggregation and anti-amplification safeguards.

(iv) *Multi-agent coordination*: Redis-backed kanban boards, workspaces, agent registration, and Pub/Sub message passing support collaborative multi-session workflows.

(v) *Provider-agnostic LLM routing*: Eight providers with native and OpenAI-compatible adapters, fallback chains, cost optimization, and Zod-schema structured output.

The remainder of this paper is organized as follows. Section II surveys related work. Section III presents the system architecture. Section IV provides a detailed technical description of all three protocol layers with formal specifications. Section V describes the LLM Magistrale. Section VI covers the cognition subsystem. Section VII catalogs the tool categories. Section VIII describes multi-agent coordination. Section IX addresses security mechanisms. Section XI presents the evaluation. Section XII concludes.

II. Related Work

The Model Context Protocol builds on the architectural patterns established by the Language Server Protocol (LSP) [4], which standardized language intelligence features across IDEs. While LSP focuses on language-specific operations (completion, diagnostics, refactoring), MCP generalizes the client-server pattern to arbitrary tool invocations for AI assistants.

Multi-agent LLM frameworks have been explored extensively in recent literature. AutoGen [12] enables multi-agent conversation through a conversable agent abstraction with customizable conversation patterns. MetaGPT [13] introduces role-based agent coordination with structured output protocols. Li et al. [10] survey workflow patterns, infrastructure requirements, and open challenges in LLM-based multi-agent systems. Guo et al. [11] provide a comprehensive taxonomy of multi-agent architectures, covering communication, memory, and planning mechanisms. KemdiCode differs from these frameworks by operating at the protocol level—it exposes coordination primitives as MCP tools rather than implementing a fixed agent architecture.

For distributed messaging, Redis Pub/Sub [5] provides the foundation for KemdiCode's inter-cluster communication, following patterns described by Lamport [15] for event ordering in distributed systems. The bloom filter [14] is employed for probabilistic deduplication in the cluster bus. Tree-sitter [8], based on the incremental parsing techniques of Wagner and Graham [7], provides the AST infrastructure. Secret sharing follows Shamir's original construction [6].

D. Irzyk, M.Sc. Eng., is with Kemdi Sp. z o.o., Poland (e-mail: dawid@kemdi.pl).

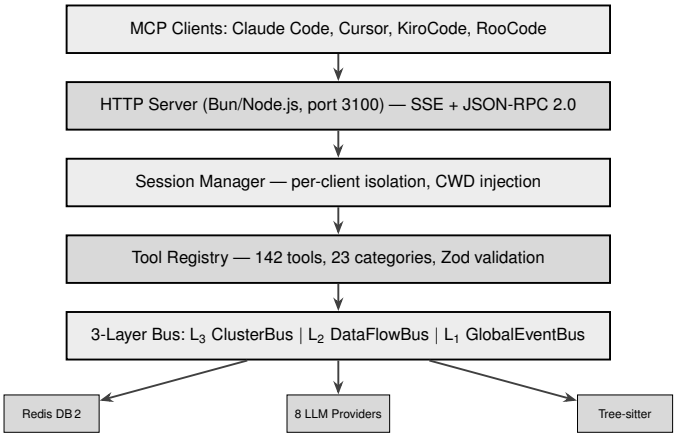

Figure 1. High-level system architecture showing the five-layer stack from client to infrastructure.

Table I
HTTP ENDPOINTS

Method	Path	Purpose
GET	/sse	SSE notification stream
POST	/message	JSON-RPC tool invocations
GET	/stream/:id	Per-request execution stream
GET	/health	Health check and uptime
GET	/tools	Tool listing with schemas
GET	/mcp	Streamable HTTP endpoint

III. SYSTEM ARCHITECTURE

Fig. 1 shows the high-level system architecture. The server is organized into five principal layers: transport, session management, tool registry, bus architecture, and shared infrastructure (including Redis, LLM providers, and Tree-sitter).

A. Transport Layer

The server implements the MCP Streamable HTTP transport, exposing the endpoints listed in Table I. Communication follows JSON-RPC 2.0 with the MCP method extensions (`tools/list`, `tools/call`, `initialize`, `notifications/initialized`, etc.). Server-Sent Events (SSE) [17] provide the server-to-client notification channel with configurable keep-alive intervals.

B. Tool Registry

The tool registry manages 142 tools through a unified `UnifiedTool` interface. Each tool defines its input schema using Zod [9], which is converted to JSON Schema for the MCP protocol. A non-trivial conversion handles Zod's `discriminatedUnion` type: since many MCP clients cannot process JSON Schema `oneOf` constructs, the registry flattens discriminated union variants into a single merged object schema via `toMcpInputSchema()`.

Tool annotations follow the MCP specification's hint system with three boolean flags: `readOnlyHint` (tool does not modify state), `destructiveHint` (tool may delete data), and `openWorldHint` (tool accesses external services). These annotations are maintained in a centralized map (`annotations-map.ts`) and attached during schema generation.

Lazy loading ensures that tool implementations are instantiated on first invocation. A background warmup phase pre-loads all schemas after server start and emits `tools/list_changed` notifications to connected clients. The availability checker reports tool health with soft and force modes, suggesting fallback alternatives when a tool's dependencies are unavailable.

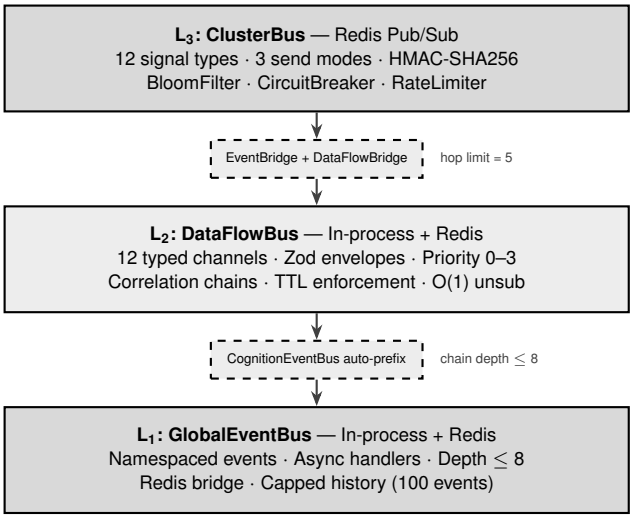

Figure 2. Three-layer bus architecture with anti-amplification bridges. Arrows indicate the primary message flow direction; bidirectional bridging is supported with hop-count guards.

IV. PROTOCOL LAYER ARCHITECTURE

The communication infrastructure consists of three layers connected by anti-amplification bridges. Each layer operates at a distinct scope and employs different transport mechanisms. Fig. 2 illustrates the layered design with bridge interconnections.

A. L_3: ClusterBus — Inter-Cluster Communication

The ClusterBus enables inter-cluster communication over Redis Pub/Sub [5]. It is the only layer that crosses process boundaries, making it the primary mechanism for distributed LLM orchestration and multi-node coordination.

1) Signal Structure: Every message on the ClusterBus is a `ClusterSignal`. The principal fields are shown below (auxiliary fields such as `correlationId`, `metaTags`, and `schemaVersion` are omitted for brevity):

```
ClusterSignal {
  id: string // UUID v4
  type: SignalType // 12 enum values
  sourceCluster: string // originating node ID
  targetCluster: string // unicast target
  payload: unknown // type-dependent data
  timestamp: number // Unix ms
  ttl: number // max hops
  priority: 0|1|2|3 // 0=lowest, 3=critical
  metadata: Record<string, unknown>
  hmac?: string // HMAC-SHA256 signature
}
```

The 12 signal types are organized into four namespaces, shown in Table II.

Table II
CLUSTERBUS SIGNAL TYPES

Namespace	Types
`llm:*`	`request`, `result`, `stream-chunk`, `error`
`cluster:*`	`join`, `leave`, `heartbeat`
`data:*`	`broadcast`, `unicast`
`control:*`	`pause`, `resume`, `config`

2) Send Modes: Three send modes are supported:
Unicast. The signal carries a `target` field specifying the destination node ID. Only the targeted node processes the signal; others discard it after deduplication.
Broadcast. The `target` field is omitted. All subscribed nodes receive and process the signal.
Routed. The signal is dispatched via the MetaRouter, which evaluates tag-based routing rules with AND/OR boolean logic. Route patterns support regex matching (capped at 200 characters to prevent ReDoS).

3) Dual-Layer Deduplication: The ClusterBus employs a dual-layer deduplication strategy to prevent duplicate signal processing:

1) *Recent set*: A bounded set of the most recent 2,000 signal IDs provides exact deduplication for recently received signals with O(1) lookup.
2) *Bloom filter*: A space-efficient probabilistic filter [14] configured for 20,000 items at a false-positive rate of 0.1%, occupying approximately 35 KB of memory. The filter uses the formula:

$$m = -\frac{n \ln p}{(\ln 2)^2}, \quad k = \frac{m}{n} \ln 2 \tag{1}$$

where n is the expected item count, p is the target false-positive rate, m is the bit array size, and k is the number of hash functions. At the configured parameters ($n = 20{,}000$, $p = 0.001$), this yields $m \approx 287{,}552$ bits and $k_{\text{opt}} \approx 10$. The implementation clamps k to the range $[2, 8]$, so $k = 8$ is used. The actual false-positive probability after inserting n' items is:

$$P(\text{FP}) = \left(1 - e^{-kn'/m}\right)^k \tag{2}$$

When the recent set reaches capacity, its oldest entries are migrated to the bloom filter before eviction. The combined false-positive rate of the dual-layer system for a signal with ID s is:

$$P_{\text{dup}}(s) = \begin{cases} 0 & \text{if } s \in R_{\text{recent}} \\ P(\text{FP}) & \text{if } s \notin R_{\text{recent}} \wedge s \notin B_{\text{actual}} \\ 0 & \text{otherwise} \end{cases} \tag{3}$$

where R_{recent} is the exact recent set and B_{actual} is the set of IDs truly present in the bloom filter. This achieves zero false positives for the 2,000 most recent signals. To prevent saturation, the bloom filter is periodically reset after processing 15,000 signals; upon reset, only the current recent-set entries are re-inserted.

4) HMAC Authentication: Signals are authenticated using HMAC-SHA256. The signing process computes a signature over the complete JSON-serialized signal:

$$\text{HMAC} = H_K\big(\text{JSON.stringify}(s)\big) \tag{4}$$

where H_K denotes HMAC-SHA256 with shared key K and s is the full signal object. Signing the entire payload rather than selected fields ensures that any tampering—including metadata modification—is detected. Verification uses a constant-time byte-by-byte XOR comparison loop to prevent timing side-channel attacks. Signals failing HMAC verification are silently dropped and logged.

5) SignalFlowController: The SignalFlowController governs the throughput and reliability of signal processing through four mechanisms:
Rate Limiting. A sliding-window rate limiter with window size $W = 1\,\text{s}$ and maximum capacity $C_{\max}$ signals per window. The effective throughput at time t is bounded by:

$$\Theta(t) \le C_{\max} \cdot \frac{1}{W} \quad \text{[signals/s]} \tag{5}$$

Excess signals are either buffered or dropped based on the configured backpressure policy.
Circuit Breaker. Implements the three-state pattern [16] modeled as a finite automaton $\mathcal{A} = (S, \Sigma, \delta, s_0)$ where $S = \{\text{closed}, \text{open}, \text{half-open}\}$, $s_0 =$ closed, and the transition function δ is defined by:

$$\delta(s, \sigma) = \begin{cases} \text{open} & s = \text{closed} \wedge f \ge F_{\max} \\ \text{half-open} & s = \text{open} \wedge \Delta t \ge T_{\text{open}} \\ \text{closed} & s = \text{half-open} \wedge p \ge P_{\min} \\ \text{open} & s = \text{half-open} \wedge \sigma = \text{fail} \end{cases} \tag{6}$$

where f is the consecutive failure count ($F_{\max} = 10$), Δt is the time since opening ($T_{\text{open}} = 5{,}000\,\text{ms}$), and p is the successful probe count ($P_{\min} = 2$).
Priority Filtering. Signals carry a priority field (0–3). Under backpressure, the controller drops low-priority signals first, ensuring critical signals (priority 3) are always processed.
Backpressure. Queue depth is monitored continuously. When the queue exceeds the high-water mark, the controller transitions to a backpressure state, activating priority filtering and signaling upstream nodes to reduce their send rate.

6) MetaRouter: The MetaRouter provides tag-based signal routing with boolean logic. Routing rules specify:

```
RouteRule {
  pattern: string // regex (max 200 chars)
  tags: string[] // required meta tags
  logic: "AND" | "OR"
  target: string // destination node/group
  priority: number
}
```

Rules are evaluated in priority order. The first matching rule determines the signal's destination. Regex patterns are length-capped to prevent ReDoS attacks.

7) HealthMonitor: Each node emits periodic heartbeat signals (type `cluster:heartbeat`). The HealthMonitor tracks:

- Last heartbeat timestamp per node.

Table III
DATAFLOWBUS CHANNELS

Domain	Channels
AI	`ai:completion`, `ai:structured`, `ai:research`
Kanban	`kanban:task-change`, `kanban:complexity`
Cognition	`cognition:decision`, `cognition:intent`, `cognition:error`
Agent	`agent:status`, `agent:message`
System	`system:health`, `system:config`

- Latency statistics (moving average).
- Stale detection: nodes exceeding the heartbeat timeout (default 15 s) are marked stale and excluded from routing decisions.
- Automatic pruning of stale entries with O(1) unsubscribe via `Map`-based tracking.

B. L_2: DataFlowBus — Inter-Module Communication

The DataFlowBus provides typed, schema-validated messaging between server modules. It operates primarily in-process but persists messages to Redis for cross-session access.

1) Channel Architecture: The bus defines 12 typed channels organized into five domains, as shown in Table III.

2) Envelope Structure: Every message is wrapped in a `DataFlowEnvelope`:

```
DataFlowEnvelope<T> {
  id: string // UUID v4
  channel: Channel // one of 12 channels
  payload: T // Zod-validated data
  correlationId: string // request chain ID
  schemaVersion: number // version counter
  priority: 0|1|2|3
  ttl: number // milliseconds
  source: string // originating module
  target?: string // optional filter
  sessionId?: string
  agentId?: string
  timestamp: number
}
```

Payloads are validated against per-channel Zod schemas at publish time.

Definition 1 (Envelope Validity): An envelope E on channel c is valid iff:

$$\mathcal{V}(E) \iff \mathcal{Z}_c(E.\text{payload}) \wedge E.\text{ttl} > 0 \wedge E.\text{ts} \leq t_{\text{now}} \quad (7)$$

where $\mathcal{Z}_c$ is the Zod schema validator for channel c.
Invalid envelopes cause synchronous rejection with descriptive error messages.

3) Subscription Management: Subscribers register for specific channels and receive messages asynchronously. The implementation uses a `Map<string, Set<Callback>>` structure providing:

- O(1) subscribe and unsubscribe operations.
- Automatic cleanup of idle subscriptions after 60 minutes of inactivity.

Algorithm 1: GlobalEventBus Dispatch

Input : event e, payload ρ, depth d

$d \leftarrow d + 1$;
if $d > D_{\max}$ **then**
  log warning, **return**;
$H \leftarrow$ handlers$[e.\text{type}]$;
sort H by priority descending;
foreach $h_i \in H$ **do**
  queueMicrotask(h_i, ρ, d);
$d \leftarrow d - 1$;

- Correlation-based filtering: subscribers may filter messages by `correlationId` to receive only messages belonging to a specific request chain.

4) Redis Persistence: Messages are persisted to Redis under the key pattern `mcp:dataflow:{channel}` as JSON-serialized envelopes. This enables cross-session message access and supports the multi-agent shared-context mechanism described in Section VIII.

C. L_1: GlobalEventBus — In-Process Events

The GlobalEventBus is a singleton, in-process event emitter with namespaced events following the pattern `module:category:action` (e.g., `cognition:decision:recorded`, `kanban:task:completed`).

1) Event Dispatch: Handlers execute asynchronously via `queueMicrotask()` with configurable priority. Algorithm 1 formalizes the dispatch procedure. The chain depth limit of 8 prevents cascading event storms where handler A emits event e_1, triggering handler B which emits e_2, and so forth. where $D_{\max} = 8$ is the maximum permitted chain depth.

2) Redis Bridge: The Redis bridge propagates selected events across sessions:

- Events matching configurable patterns are serialized and published to Redis Pub/Sub channels under `mcp:events:{type}`.
- Incoming events from other sessions are deserialized and re-emitted on the local bus with a `_remote` flag to prevent re-publication.
- Failed publications are retried twice with exponential backoff (100 ms, 200 ms).
- Event history is capped at 100 entries per type with a 1-hour TTL in Redis.

D. Anti-Amplification Bridge Mechanism

Bridges connect the three layers, enabling signals originating at one scope to propagate to another. Without safeguards, bridges would create feedback loops: a signal bridged from L_3 to L_1 could trigger an event that bridges back to L_3, creating infinite recursion.

Four anti-amplification mechanisms prevent this:

Invariant 1 (Signal Termination): For any signal s entering the bus system, the total number of copies produced across all layers is bounded by:

$$|C(s)| \leq H_{\max} \cdot D_{\max} = 5 \times 8 = 40 \quad (8)$$

Table IV
ANTI-AMPLIFICATION GUARDS

Guard	Mechanism
Hop counter	Signals carry `_bridgeHops`, incremented per crossing. Max = 5.
Source prefix	Skip signals whose `source` starts with the bridge's own layer prefix.
Dedup set	Per-bridge message-ID set prevents re-bridging.
Chain depth	L_1 enforces depth ≤ 8, limiting cascades.

Table V
BRIDGE SIGNAL MAPPINGS

Direction	Source Signal	Target
$L_3 \to L_2$	`llm:result`	`ai:completion`
$L_3 \to L_2$	`data:*`	`agent:message`
$L_2 \to L_3$	`ai:research`	`llm:request`
$L_3 \to L_1$	`cluster:join`	`system:node-joined`
$L_1 \to L_3$	`system:alert`	`control:config`

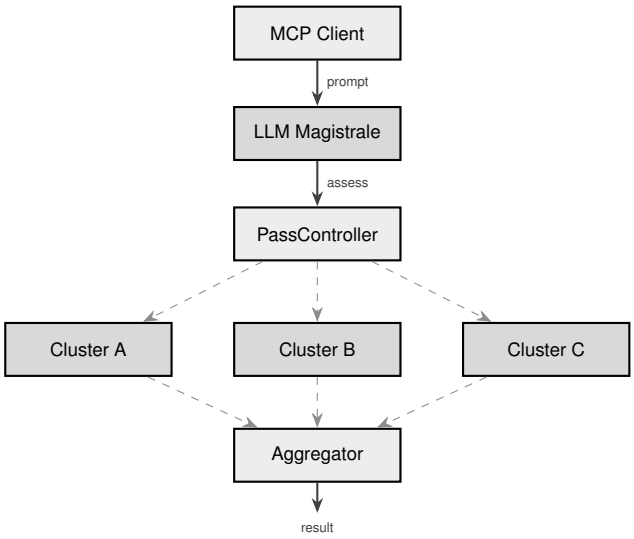

Figure 3. LLM Magistrale dispatch flow. Solid arrows indicate control flow; dashed arrows indicate asynchronous signal-based communication via the ClusterBus.

where $H_{\max}$ is the bridge hop limit and $D_{\max}$ is the event chain depth limit. In practice, deduplication reduces this to $|C(s)| \leq 3$ (one per layer).

The bridge mapping is summarized in Table V.

V. LLM MAGISTRALE

The LLM Magistrale orchestrates distributed prompt execution across multiple clusters, each potentially backed by a different LLM provider. Fig. 3 illustrates the dispatch flow.

A. Aggregation Strategies

The Magistrale supports four aggregation strategies, selectable per request:

First-wins. Returns the first response received, canceling remaining cluster requests. Optimizes for latency.

Best-of-n. Collects all n responses and scores each using the composite scoring function (Section V-B). Returns the highest-scoring response.

Consensus. Computes pairwise weighted Jaccard similarity between all responses. Requires agreement above a configurable threshold (default 0.6). If no consensus is reached, falls back to best-of-n.

Fallback-chain. Tries clusters sequentially in priority order. Returns the first successful response; continues to the next cluster on failure.

B. Composite Scoring Function

The best-of-n and consensus strategies evaluate responses using a weighted composite score:

$$S = w_q \cdot Q + w_d \cdot D + w_t \cdot T - w_l \cdot L \tag{9}$$

where:

- $Q \in [0, 1]$: Quality score assessed by the LLM self-evaluation.
- $D = \min(1, \log_2(\text{depth} + 1) / \log_2(\text{maxDepth} + 1))$: Reasoning depth, normalized on a logarithmic scale.
- $T = \min(1, |\text{tokens}| / \text{targetTokens})$: Token utilization ratio.
- $L = \min(1, \text{latency}/\text{timeout})$: Latency penalty.

The default weights are $w_q = 0.45$, $w_d = 0.25$, $w_t = 0.15$, $w_l = 0.15$, reflecting the prioritization of response quality over depth and efficiency.

C. Consensus Algorithm

For the consensus strategy, pairwise similarity is computed using a weighted Jaccard coefficient over token sets:

$$J_w(A, B) = \frac{\sum_{t \in A \cap B} \min(w_A(t), w_B(t))}{\sum_{t \in A \cup B} \max(w_A(t), w_B(t))} \tag{10}$$

where $w_X(t)$ is the TF-IDF weight of token t in response X. The consensus result is the response with the highest average pairwise similarity, provided it exceeds the agreement threshold.

D. PassController

The PassController governs multi-pass LLM execution within a single cluster, enabling iterative refinement of responses.

1) Strategies: Three strategies are supported:

Min-passes. The LLM self-assesses task complexity and declares a minimum pass count, capped to the `maxPasses` budget. Early termination occurs when quality exceeds the target.

Quality-target. Iterates pass i until convergence. The termination condition is:

$$Q_i \geq Q_{\text{target}} \quad \vee \quad i \geq N_{\max} \tag{11}$$

where Q_i is the quality score after pass i and $N_{\max}$ is the budget. Each pass receives the previous pass's output as context, forming the recurrence $Q_{i+1} = f(Q_i, \text{context}_i)$.

Fixed. Executes exactly N passes without assessment. Used for deterministic pipelines.

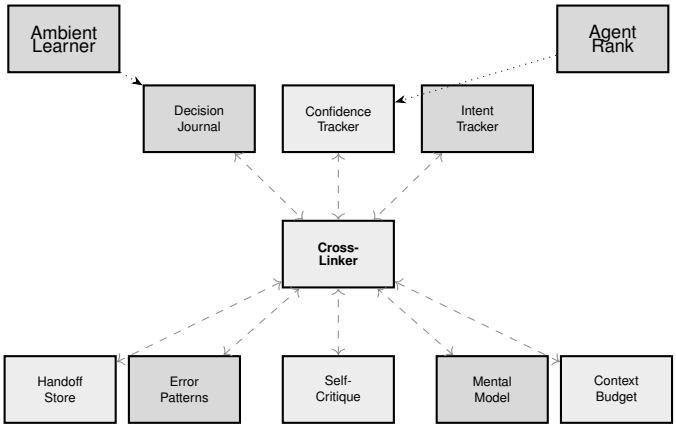

Figure 4. Cognition stores interconnected via CognitionCrossLinker. Dashed arrows: Redis cross-links. Dotted arrows: auxiliary feeds.

2) Assessment Bypass: To reduce latency for simple queries, the PassController applies bypass heuristics before invoking the LLM assessment:

- Prompts shorter than 100 tokens bypass assessment and execute a single pass.
- Prompts containing explicit single-step indicators (e.g., "translate", "summarize") bypass assessment.
- Previously-assessed prompts with similar fingerprints reuse cached pass counts.

VI. Cognition Subsystem

The cognition layer provides persistent AI self-awareness across sessions through eight primary stores, two auxiliary analyzers, and a cross-linking mechanism, all backed by Redis with configurable TTLs. Fig. 4 shows the store interconnections.

A. Primary Stores

1) DecisionJournal: Records decisions with structured fields: question, options array with pros/cons, chosen option, reasoning, confidence level, and optional outcome. Supports retrospective evaluation where outcomes are attached after the decision's consequences are observed. Stored at `mcp:cognition:decisions:{id}`.

2) ConfidenceTracker: Maintains calibrated confidence scores in the range $[0, 1]$ per domain. Calibration quality is measured by the Expected Calibration Error over a sliding window of N predictions:

$$\text{ECE} = \sum_{b=1}^{B} \frac{|n_b|}{N} \left|\text{acc}(b) - \text{conf}(b)\right| \tag{12}$$

where predictions are partitioned into B bins, n_b is the bin count, $\text{acc}(b)$ is the observed accuracy, and $\text{conf}(b)$ is the mean predicted confidence. Trend data (improving, stable, declining) is derived from the ECE gradient over the window. Stored at `mcp:cognition:confidence:{domain}`.

3) IntentTracker: Implements hierarchical goal tracking with three levels: mission (top-level objective), goals (concrete milestones), and sub-goals. Drift detection triggers when the current activity diverges from declared goals by more than a configurable threshold. Stored at `mcp:cognition:intents:{id}`.

4) ErrorPatternDB: A cross-session error database that stores error signatures (type, message pattern, stack fingerprint) with associated context, resolution steps, and recurrence counts. New errors are matched against known patterns using symptom similarity scoring. Stored at `mcp:cognition:errors:{hash}`.

5) SelfCritiqueStore: Produces structured reflections after task completion. Each critique contains: what went well, what could improve, lessons learned, and action items. Supports check-application queries that retrieve relevant lessons for a given context. Stored at `mcp:cognition:critiques:{id}`.

6) MentalModelStore: Maintains component-relationship maps representing the agent's understanding of system architecture. Models include: components with descriptions, relationships with types (depends-on, contains, communicates-with), dependency chains, and invariant assertions. Supports impact analysis: given a proposed change, enumerate affected components. Stored at `mcp:cognition:models:{name}`.

7) HandoffStore: Produces auto-enriched session snapshots for structured context transfer between agents. A handoff report includes: active task summary, key decisions made, files modified, open questions, and recommendations. Enrichment automatically attaches recent cognition state (confidence levels, intent tree, relevant error patterns). Stored at `mcp:cognition:handoffs:{id}`.

8) ContextBudgetManager: Estimates token consumption for context windows using the approximation of 1 token $\approx$ 4 characters. The manager computes a relevance score for each context item using a weighted average:

$$R = w_r \cdot \text{recency} + w_f \cdot \text{frequency} + w_i \cdot \text{importance} \tag{13}$$

with default weights $w_r = 0.4$, $w_f = 0.3$, $w_i = 0.3$. Items below the relevance threshold are suggested for trimming.

B. Auxiliary Components

1) AmbientLearner: Silently gathers knowledge from tool usage patterns without explicit invocation:

- *Tool sequences*: Records ordered tool invocation chains, identifying common workflows (e.g., `git-diff` $\rightarrow$ `code-review` $\rightarrow$ `fix-bug`).
- *File relationships*: Tracks which files are frequently accessed together, building an implicit dependency graph.
- *Time patterns*: Records temporal patterns (tool usage frequency by time of day, session duration distributions).

2) AgentRankStore: Maintains a composite agent ranking system with five tiers: bronze (0–200), silver (201–400), gold (401–600), platinum (601–800), and diamond (801–1000). The composite score is computed as:

$$\text{Score} = 0.4 \cdot S_r + 0.3 \cdot C_t + 0.2 \cdot A_s + 0.1 \cdot C_o \tag{14}$$

where S_r is the success rate, C_t is task complexity (normalized), A_s is average speed (inverse-normalized), and C_o is collaboration score. To emphasize recent performance, scores undergo exponential temporal decay:

$$\text{Score}(t) = \text{Score}_0 \cdot \alpha^{\Delta t/\tau} \tag{15}$$

where $\alpha = 0.95$ is the decay factor, Δt is the elapsed time since the last activity, and $\tau = 1\,\text{day}$ is the decay period. The tier mapping is defined as:

$$\text{Tier}(s) = \begin{cases} \text{diamond} & s \in [801, 1000] \\ \text{platinum} & s \in [601, 800] \\ \text{gold} & s \in [401, 600] \\ \text{silver} & s \in [201, 400] \\ \text{bronze} & s \in [0, 200] \end{cases} \tag{16}$$

C. CognitionCrossLinker

The CrossLinker maintains bidirectional Redis links between store entries. When a decision is recorded, the CrossLinker creates links to associated confidence scores, intent nodes, and error patterns. Links are stored as Redis sets at `mcp:cognition:links:{source}` and `mcp:cognition:links:{target}`, enabling both forward and reverse traversal.

D. Reactive Event Handlers

Nine reactive handlers on the GlobalEventBus create a feedback loop between cognition stores:

1) `decision:recorded` → Auto-create confidence record and cross-link.
2) `confidence:low` → Check intent drift.
3) `error:recorded` → Scan recent decisions for causal links.
4) `error:matched` → Log known fix suggestion.
5) `critique:lesson-learned` → Link to matching error patterns.
6) `intent:drifted` → Trigger self-critique.
7) `handoff:created` → Link relevant mental models.
8) `model:stale` → Flag affected decisions and intents.
9) `confidence:outcome` → Propagate to linked decisions.

VII. Tool Categories

Table VI enumerates all 23 tool categories with counts and representative tools.

Table VI
Tool Categories (142 Total)

Category	#	Key Tools
Cluster Bus	7	`bus-status`, `-magistrale`
Cognition	8	`decision-journal`, `confidence`
AI Agents	4	`plan`, `build`, `brainstorm`
Multi-LLM	3	`multi-prompt`, `consensus`
Code Analysis	8	`code-review`, `find-def`
Line Editing	4	`insert-at-line`, `replace-lines`
Symbol Editing	3	`rename-symbol`, `insert-after`
Code Mod.	5	`fix-bug`, `refactor`, `auto-fix`
Project Memory	8	`write-memory`, `checkpoint-save`
Git	8	`git-status`, `git-diff`, `git-log`
File Operations	9	`file-read`, `file-write`, `file-search`
Project	5	`run-tests`, `run-lint`, `check-types`
Kanban Tasks	12	`task-create`, `task-claim`
Kanban Workspaces	5	`workspace-create`, `-join`
Kanban Boards	6	`board-create`, `board-share`
Recursive	4	`invoke-tool`, `orchestrate`
Multi-Agent	14	`agent-register`, `shared-thoughts`
Pipeline	1	`pipeline`
Session	6	`session-recover`, `session-create`
MCP Client	3	`client-sampling`, `client-elicit`
Knowledge Graph	4	`graph-query`, `loci-recall`
Thinking	1	`thinking-chain`
MPC Security	4	`mpc-split`, `mpc-reconstruct`
RL Learning	2	`rl-reward-stats`
System	8	`env-info`, `tool-health`, `ping`

VIII. Multi-Agent Coordination

KemdiCode supports multi-agent workflows where multiple AI sessions collaborate on shared tasks through four mechanisms:

Agent Registry. Agents register with role (supervisor, worker, observer), capabilities, and status. Discovery is via `agent-list`; real-time monitoring via `agent-watch`.

Message Passing. `agent-inject` sends context to specific agents via Redis Pub/Sub (`inject:{agentId}`). `agent-alert` broadcasts to all agents on the `alerts` channel. `shared-thoughts` provides a collective knowledge base accessible by all agents.

Kanban System. Hierarchical task management: workspaces contain boards, boards contain tasks. Tasks support claiming, assignment, comments, and status transitions (`pending` → `in_progress` → `completed`). Role-based membership (owner, admin, member, viewer) controls access at the board level.

Task Intelligence. `task-cluster` uses LLM-driven semantic clustering to group related tasks. `task-complexity`

provides AI-powered difficulty estimation with priority scoring.

IX. Security

A. Input Validation

All tool inputs are validated via Zod schemas before execution. File paths undergo traversal protection through `validatePathSafe()`, which rejects paths containing `..` traversal sequences, null bytes, and paths outside the session's working directory.

B. Redis Security

JSON parsing from Redis employs prototype pollution protection, blocking dangerous keys: `__proto__`, `constructor`, and `prototype`. This prevents injection of properties into JavaScript's object prototype chain.

C. Cluster Bus Authentication

As described in Section IV-A, all cluster signals are HMAC-SHA256 authenticated with constant-time comparison. The shared key is configured at server start and is never transmitted over the bus.

D. Multi-Party Computation

Shamir's Secret Sharing [6] enables distributed secret management. A secret s is encoded as the constant term of a random polynomial of degree $t-1$ over a finite field $\mathbb{F}_p$:

$$f(x) = s + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \pmod{p} \quad (17)$$

where $a_1, \ldots, a_{t-1} \xleftarrow{\$} \mathbb{F}_p$. Each of the n shares is a point $(i, f(i))$ for $i \in \{1, \ldots, n\}$. Reconstruction from any t shares uses Lagrange interpolation to recover $f(0) = s$. The tools `mpc-split`, `mpc-distribute`, and `mpc-reconstruct` implement this protocol.

E. Anti-ReDoS

MetaRouter regex patterns are capped at 200 characters, preventing Regular Expression Denial of Service attacks through pathological backtracking.

X. LLM Provider Integration

KemdiCode integrates eight LLM providers through a unified routing layer with the model specification syntax `provider:model:thinking`. Table VII summarizes the supported providers.

The routing layer provides three-tier model selection: primary model for standard requests, research model for complex analysis, and fallback model for quota exhaustion or provider errors. Provider connections are lazily initialized and support hot-reload without server restart.

Structured output is produced via `generateObject()`, which combines Zod schemas with `jsonrepair` to handle malformed LLM JSON output. This provides reliable typed data extraction across all providers regardless of their native structured output support.

Table VII
LLM Providers

Provider	Alias	SDK	Notes
Anthropic	`a`	Native	Extended thinking
OpenAI	`o`	Native	Structured output
Gemini	`g`	Native	Flash/Pro/Ultra
Groq	`q`	Compat.	Fast inference
DeepSeek	`d`	Compat.	V3, R1
Ollama	`l`	Compat.	Local models
OpenRouter	`r`	Compat.	100+ models
Perplexity	`p`	Compat.	Search-augmented

XI. Evaluation

A. Test Suite

The system is validated by a comprehensive test suite of 559 passing tests covering all tool categories, bus layers, cognition stores, and edge cases. Tests are executed via `bun test` with the Bun runtime.

B. Client Compatibility

KemdiCode MCP Server has been verified with four MCP clients: Claude Code (primary), Cursor, KiroCode, and RooCode. All clients successfully discover tools, invoke them, and receive SSE notifications without modification.

C. Infrastructure

The server supports two runtimes (Bun and Node.js) through a runtime abstraction layer that provides unified interfaces for HTTP serving, process management, and cryptographic operations. Tree-sitter [8] WASM parsers provide AST-based code analysis for 19 programming languages.

All persistent state uses Redis DB 2 with 10 key namespaces (`mcp:context:*`, `mcp:agents:*`, `mcp:messages:*`, `mcp:kanban:*`, `mcp:memory:*`, `mcp:cognition:*`, `mcp:cluster:*`, `mcp:dataflow:*`, `mcp:events:*`, `mcp:channel:*`).

D. Performance Characteristics

- Lazy schema loading eliminates startup latency for unused tools.
- O(1) subscribe/unsubscribe operations on both DataFlowBus and HealthMonitor via `Map`-based tracking.
- Circuit breakers in the SignalFlowController prevent cascade failures.
- Bloom filter deduplication uses 35 KB for 20,000-item capacity at 0.1% false-positive rate.
- Idle subscription cleanup after 60 minutes prevents resource leaks.

XII. Conclusion

KemdiCode MCP Server v1.25.0 demonstrates that the Model Context Protocol can serve as a foundation for comprehensive AI-assisted software engineering platforms. By combining 142 specialized tools, a three-layer distributed bus architecture with formalized anti-amplification guards,

a persistent cognition system with reactive cross-linking, and multi-provider LLM orchestration with quality-driven aggregation, the server transforms MCP from a simple tool-calling interface into a stateful, distributed AI development infrastructure.

The three-layer bus design (L_3 ClusterBus, L_2 DataFlowBus, L_1 GlobalEventBus) provides clear separation of concerns: inter-cluster communication with cryptographic authentication, typed inter-module messaging with schema validation, and efficient in-process event propagation with cascade protection. The anti-amplification bridge mechanism, formalized through hop counters, source guards, deduplication sets, and chain-depth limits, ensures system stability under complex multi-layer message flows.

The architecture draws on established distributed systems techniques [14], [15] and recent advances in LLM-based multi-agent coordination [10], [11]. Future work includes expanding the knowledge graph with cross-project learning, adding WebSocket transport support, and deepening the reinforcement learning subsystem for autonomous tool selection optimization.

REFERENCES

[1] Anthropic, "Introducing the Model Context Protocol," Anthropic Blog, Nov. 2024. [Online]. Available: https://www.anthropic.com/news/model-context-protocol

[2] Model Context Protocol Authors, "Model Context Protocol Specification, version 2025-11-25," 2025. [Online]. Available: https://modelcontextprotocol.io/specification/2025-11-25

[3] JSON-RPC Working Group, "JSON-RPC 2.0 Specification," 2010. [Online]. Available: https://www.jsonrpc.org/specification

[4] Microsoft, Red Hat, and Codenvy, "Language Server Protocol Specification," 2016. [Online]. Available: https://microsoft.github.io/language-server-protocol/

[5] S. Sanfilippo, "Redis: An in-memory data structure store," 2009–2026. [Online]. Available: https://redis.io/

[6] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, pp. 612–613, Nov. 1979. doi:10.1145/359168.359176

[7] T. A. Wagner and S. L. Graham, "Efficient and flexible incremental parsing," *ACM Trans. Program. Lang. Syst.*, vol. 20, no. 5, pp. 980–1013, Sep. 1998. doi:10.1145/293677.293678

[8] M. Brunsfeld *et al.*, "Tree-sitter: An incremental parsing system for programming tools," 2018–2026. [Online]. Available: https://tree-sitter.github.io/

[9] C. McDonnell, "Zod: TypeScript-first schema validation with static type inference," 2020–2026. [Online]. Available: https://zod.dev/

[10] J. Li, Q. Wang *et al.*, "A survey on LLM-based multi-agent systems: Workflow, infrastructure, and challenges," *Vicinagearth*, Oct. 2024. doi:10.1007/s44336-024-00009-2

[11] T. Guo, X. Chen *et al.*, "Large language model based multi-agents: A survey of progress and challenges," in *Proc. 33rd Int. Joint Conf. Artif. Intell. (IJCAI)*, 2024.

[12] Q. Wu, G. Bansal *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," *arXiv:2308.08155*, 2023.

[13] S. Hong, X. Zhuge *et al.*, "MetaGPT: Meta programming for multi-agent collaborative framework," *arXiv:2308.00352*, 2023.

[14] B. H. Bloom, "Space/time trade-offs in hash coding with allowable errors," *Commun. ACM*, vol. 13, no. 7, pp. 422–426, Jul. 1970. doi:10.1145/362686.362692

[15] L. Lamport, "Time, clocks, and the ordering of events in a distributed system," *Commun. ACM*, vol. 21, no. 7, pp. 558–565, Jul. 1978. doi:10.1145/359545.359563

[16] M. T. Nygard, *Release It! Design and Deploy Production-Ready Software*. Raleigh, NC, USA: Pragmatic Bookshelf, 2007.

[17] I. Hickson, "Server-Sent Events," W3C Recommendation, Feb. 2015. [Online]. Available: https://www.w3.org/TR/eventsource/